

[← Back to CCDV-F Sample Tests](#)

Great Job!

Model Selection and Optimization Practice Test Complete

9/10

Correct

90%

Score

16:00

Time



This was a sample test from a limited pool of questions.
Upgrade to Premium for the complete CCDV-F bank with hundreds more questions!

Upgrade to Premium

Get unlimited access to:

- ✓ Unlimited practice tests across all domains
- ✓ Full-length 53-question mock exams with realistic timing
- ✓ Detailed explanations and study guides
- ✓ Progress tracking and performance analytics
- ✓ Smart practice focusing on weak areas

Review Your Answers

1

[→ Login to Clarify](#)

A team is adding a Claude-powered intent router in front of a support system. The task is to map each

message to one of 12 categories and return a short JSON label. A validation set shows simple examples with little ambiguity.

Artifact: Workload: 3 million requests/month Target: p95 latency under 700 ms Business priority: lowest cost that still passes validation Quality bar: 95% label accuracy on held-out examples Current plan: use the most capable model for all requests

What is the best initial model-selection approach?

- A.
Start with Haiku and validate accuracy before rollout.



- B.
Use Opus because it provides the highest capability.

- C.
Use extended thinking for every routing request.

- D.
Use multi-shot examples with the largest context.

Explanation:

Correct answer (A): For a high-volume, low-latency, cost-sensitive classification or routing task, Haiku is generally the best starting point when validation can confirm it meets the quality bar. The scenario describes a simple constrained-label task, a strict latency target, and a cost priority, so the developer should test the smaller/faster model rather than defaulting to a more expensive model or higher reasoning configuration.

Why the other options are wrong:

- Option B: Opus may be attractive because it is the most capable model, but the task is simple and highly cost- and latency-sensitive. It is not the best initial choice when a smaller model can be validated.

- Option C: Extended thinking can help complex reasoning tasks, but this router only needs simple classification. It would add latency and token use without matching the stated priority.
- Option D: Examples can improve behavior, but using many examples and the largest context increases token usage. The stem asks for model selection under a low-cost, low-latency goal.

2 [→ Login to Clarify](#)

A developer is building an internal contract review assistant. It summarizes clauses and flags unusual obligations for a legal operations team, but lawyers still review the final output.

Artifact: Quality need: strong reasoning over long legal text Latency need: interactive responses under a few seconds when possible Budget: moderate; cannot use the highest-cost path for every request Risk: important, but not autonomous final decision-making

Which model strategy is most appropriate as the default?

- A.
Use Sonnet as the default and evaluate edge cases.



- B.
Use Haiku for all reviews to minimize cost.

- C.
Use Opus for every clause regardless of complexity.

- D.
Use fast mode only and skip quality evaluation.

Explanation:

Correct answer (A): Sonnet is commonly the balanced default when an application needs strong quality while preserving better latency and cost than the most capable model. The task requires meaningful reasoning over legal text, so the smallest model for all cases is risky, but the scenario does not justify using the most expensive model for every request because lawyers review outputs and budget is moderate.

Why the other options are wrong:

- Option B: Haiku may reduce cost, but the scenario calls for strong reasoning over legal language. Choosing it for all reviews without validation would underweight quality requirements.
- Option C: Opus may help the hardest cases, but the scenario states a moderate budget and interactive latency needs. Using it for every clause is not the best default.
- Option D: Fast mode may improve latency, but skipping quality evaluation is inappropriate for a legal-review workflow with stated reasoning requirements.

3 

[→ Login to Clarify](#)

A SaaS product uses Claude to answer questions about a customer's 80-page policy manual. Each request sends the same system instructions and the same manual, followed by a short user question.

Artifact: Per request input: 58,000 stable tokens + 80 changing tokens Request pattern: 40,000 requests/day per large customer Manual update frequency: once per quarter Current issue: input-token cost is much higher than expected

What is the best primary optimization?

- A.
Use prompt caching for the stable manual prefix.



B.
Switch every request to Opus for better quality.

C.
Increase output token limits for all answers.

D.
Move the changing question into the cached prefix.

Explanation:

Correct answer (A): Prompt caching is most useful when many requests share a large, stable prefix or reusable context. Here, almost all input tokens come from the same manual and instructions repeated across many requests, while only a small user question changes. Caching the stable prefix can reduce repeated processing cost when the prompt is structured consistently around stable boundaries.

Why the other options are wrong:

- Option B: A larger model might improve quality in some cases, but the problem described is repeated input-token cost from stable context. Switching to Opus does not address that cost driver.
- Option C: Increasing output limits could raise cost and latency. The artifact shows the main issue is large repeated input, not insufficient answer length.
- Option D: Frequently changing content should not be placed in the cacheable prefix because it prevents cache reuse. The user question should remain outside the stable cached portion.

4 

[→ Login to Clarify](#)

A developer wants to cache prompt content for a Claude-powered analytics assistant. The prompt is

assembled in this order:

Artifact:

1. System instructions, tool-use policy, and output format: changes monthly
2. Company data dictionary: changes monthly
3. User's selected dashboard filters: changes every request
4. Retrieved rows for the current question: changes every request
5. User question: changes every request

The goal is to maximize cache reuse without serving stale request-specific context. Where should the cache checkpoint be placed?

- A.
After the company data dictionary section.



- B.
After the selected dashboard filters section.

- C.
After the retrieved rows for the question.

- D.
After the final user question section.

Explanation:

Correct answer (A): Cache checkpoints should align with stable prompt boundaries. The system instructions and data dictionary change infrequently and are reused across requests, while filters, retrieved rows, and the user question change every request. Placing the checkpoint after the data dictionary maximizes reusable stable content while keeping request-specific content outside the cached prefix.

Why the other options are wrong:

- Option B: Filters change every request, so including them in the cached prefix would reduce cache reuse and risk tying the cache to request-specific content.
- Option C: Retrieved rows are request-specific and change every request. Placing the checkpoint there would make the prefix unstable and unlikely to be reused.
- Option D: The final user question changes every request. Caching through it would defeat the purpose of a reusable prefix.

5



→ Login to Clarify

A product feature asks Claude to answer employee benefits questions. Most questions are simple policy lookups, but about 10% require comparing several policy sections and resolving exceptions.

Artifact: Current setting: fixed high reasoning effort for every request
Issue: simple questions feel slow and cost more than budgeted
Quality issue: complex exception questions need deeper reasoning
Goal: reduce unnecessary latency while preserving quality on hard cases

Which runtime strategy is the best fit?

A.

Use adaptive thinking so effort varies by complexity.



B.

Use fixed extended thinking for every request.

C.

Disable reasoning effort for every request.

D.

Move all requests to Haiku without evaluation.

Explanation:

Correct answer (A): Adaptive thinking is useful when task complexity varies and the application benefits from allowing the model to spend more effort only when needed. The current fixed high-effort setting over-optimizes simple lookups, increasing latency and cost, while the hard exception cases still need deeper reasoning. Adaptive effort better matches the mixed workload.

Why the other options are wrong:

- Option B: Fixed extended thinking can help hard cases, but it repeats the current problem by adding unnecessary cost and delay to simple questions.
- Option C: Disabling reasoning effort for all requests may improve speed but ignores the stated need for deeper reasoning on complex exception questions.
- Option D: Haiku could be evaluated for simple lookups, but switching all traffic without evaluation does not address the mixed complexity and quality requirement.

6



[→ Login to Clarify](#)

A CI test calls Claude with the same prompt and compares the full response string to a stored golden response. The test is failing intermittently even though the answers remain semantically correct.

Artifact: Run 1: "The renewal notice must be sent 30 days before expiration." Run 2: "Send the renewal notice at least 30 days before the policy expires." Run 3: "The notice deadline is 30 days before expiration."

What is the best adjustment to the test strategy?

- A.
Evaluate required meaning instead of exact wording.



- B.
Treat any wording difference as model failure.

- C.
Switch to WebSockets to force identical output.

- D.
Increase context size until responses match exactly.

Explanation:

Correct answer (A): LLM outputs can vary across runs because generation is probabilistic and based on next-token prediction and sampling. For this type of CI check, exact string comparison is brittle when semantically correct answers can be phrased differently. The better test verifies required meaning, structured fields, or acceptance criteria rather than identical prose.

Why the other options are wrong:

- Option B: Exact wording comparison may be appropriate for deterministic code paths, but it is brittle for probabilistic model output when the meaning is correct.
- Option C: WebSockets or streaming-oriented transports affect response delivery and perceived responsiveness. They do not make model generation identical across runs.
- Option D: A larger context window does not guarantee identical wording. It can also increase cost and latency without fixing the test-design issue.

A developer increased the amount of retrieved context sent to Claude for a knowledge-base assistant after moving to a model with a larger context window.

Artifact: Before: 8 retrieved passages, average answer latency 2.1 s, good relevance After: 80 retrieved passages, average answer latency 7.8 s, answers cite less relevant passages Cost: input token spend increased 9x Goal: improve answer quality while reducing wasted tokens

What is the best next optimization?

A.

Prune retrieved context to the most relevant passages.



B.

Keep all passages because larger context always helps.

C.

Add more few-shot examples before the passages.

D.

Raise output length so citations include everything.

Explanation:

Correct answer (A): A larger context window permits more material, but it does not mean more context always improves quality. Irrelevant or excessive context can increase cost and latency and can reduce focus. The artifact shows higher latency, higher input cost, and poorer citation relevance after adding many passages, so pruning to the most relevant context is the best optimization.

Why the other options are wrong:

- Option B: The larger context window makes inclusion possible, but the observed results show more context is hurting cost, latency, and focus.
- Option C: Few-shot examples can guide behavior, but they also add tokens and do not

address the main problem of too much irrelevant retrieved content.

- Option D: Raising output length may increase cost and verbosity. It does not improve the relevance of input context or citations.

8

[→\] Login to Clarify](#)

A production summarization service uses an unpinned model alias. After a routine dependency deploy, support tickets report that summaries are shorter and omit required risk notes. No prompt or application code changed.

Artifact: Deployment diff: SDK patch version updated Model parameter: "latest-sonnet" Regression: output style changed on 18% of evaluation cases SLO: stable summary format for regulated reports

What is the best immediate mitigation?

A.

Pin a known model version and rerun evaluations.



B.

Increase max output tokens for all summaries.

C.

Switch to Haiku to reduce production cost.

D.

Accept the new behavior from the alias.

Explanation:

Correct answer (A): Production applications that require stable behavior should pin model versions or otherwise control upgrades, because model releases can introduce behavior changes. The artifact shows an unpinned alias and regression in evaluation cases without prompt or app-code changes. Pinning a known version and rerunning evaluations is the safest immediate mitigation before adopting any changed behavior.

Why the other options are wrong:

- Option B: More output tokens might reduce truncation, but the artifact points to changed model behavior from an unpinned alias, not an output-length limit.
- Option C: Switching to Haiku optimizes cost but is unrelated to restoring regulated summary behavior and may reduce quality for this task.
- Option D: Accepting alias behavior ignores the stated SLO for stable regulated reports and skips evaluation of the behavior change.

9

[→ Login to Clarify](#)

A developer is optimizing a Claude prompt for extracting invoice fields into JSON. Zero-shot extraction misses vendor aliases, while adding examples improves accuracy but increases token usage.

Artifact: Variant A: zero-shot, 1,000 input tokens, 88% valid extraction
Variant B: one example, 1,650 input tokens, 94% valid extraction
Variant C: six examples, 4,900 input tokens, 95% valid extraction
Target: at least 93% valid extraction at lowest practical token cost

Which prompt variant is the best choice?

- A.
Variant B with one representative example.



- B.
Variant A because zero-shot uses fewer tokens.

C.

Variant C because more examples are always best.



D.

Use no examples and raise output token limits.

Explanation:

Correct answer (A): Single-shot prompting uses one example and can improve model behavior while adding fewer tokens than multi-shot prompting. Variant B is the lowest-token option that meets the stated 93% validity target. Variant C improves accuracy only slightly beyond B while using far more input tokens, and Variant A fails the target.

Why the other options are wrong:

- Option B: Zero-shot has the lowest token usage, but it does not meet the required validity target in the artifact.
- Option C: Multi-shot examples can improve behavior, but more examples increase token usage. Here the small gain over Variant B does not justify the much larger token cost.
- Option D: Raising output token limits does not address missed vendor aliases in extraction behavior and can increase cost.

10

[→ Login to Clarify](#)

A team wants to cut Claude API spend for a product-description generator without reducing accepted output quality. They collected one week of usage data.

Artifact: Model: Sonnet Input tokens/request: 2,200 average Output tokens/request: 1,900 average Rejected outputs: 3% Prompt structure: 12 style examples repeated in every request Observation: accepted descriptions average 650 tokens, but max output is set high

What is the best first optimization to investigate?

- A.
Reduce requested output length and track acceptance.



- B.
Switch all traffic to Opus for fewer rejections.

- C.
Remove usage tracking and rely on invoices.

- D.
Add more examples to improve style adherence.

Explanation:

Correct answer (A): Token usage tracking should guide optimization decisions. The artifact shows output tokens are a major cost driver, accepted descriptions are much shorter than the generated average, and rejection rate is already low. Reducing output length while tracking acceptance directly targets wasted output tokens without immediately changing model quality. Further prompt or model changes may be evaluated later, but this is the best first investigation from the data.

Why the other options are wrong:

- Option B: Opus may reduce failures in some difficult tasks, but rejection rate is already low and switching up would likely increase cost rather than address excess output length.
- Option C: Removing tracking would make optimization less informed. The scenario depends on token metrics to identify where spend is going.
- Option D: More examples can improve style, but examples add input tokens. The data shows output length is a prominent cost issue and acceptance is already high.

